ASSIGNMENT 4 - CO4

Designing Reusable Class Hierarchies

S Vinoth Kumar

Register number: 192471006

Object Oriented Programming with C++ (DSA01)

Course outcome CO4: inheritance, composition, constructors, pointers and polymorphism.

GitHub repository

https://github.com/vinoththecool7-netizen/DSA01-192471006-Assignment4

Six tasks | Executed C++17 programs | Design and analysis report

Execution method and evidence

Compiler: Apple Clang 21.0.0 (clang-2100.4.1.1), arm64 Apple Darwin, invoked as g++. This is the installed Apple compiler alias, not GNU GCC. Every program uses -std=c++17 -Wall -Wextra; sanitizer variants additionally use -fsanitize=address -g. Exact relative commands, source SHA-256 digests, process statuses and consumed inputs are recorded alongside each output.

The required predictions were saved before the first execution in this workflow. prediction_record.json records the actual sequence and hashes; it does not assert prior personal student activity. All supplied prediction outputs and the two completed TODO-program outputs matched byte for byte, including significant trailing spaces and final newlines.

Embedded data is identified for Tasks 1-5; Task 6 reads Task06/input.txt. Output screenshots are genuine browser captures of the archived execution-record viewer, labelled as such. They are not imitation terminal images. Text logs remain the authoritative complete evidence. The only expected ordinary compiler warning is the deliberately reordered initializer-list fixture; the ambiguous-member fixture is an expected compile failure.

ASan limitations

The Mac supports AddressSanitizer but rejects detect_leaks=1. The broken account program exits 0 under the ordinary ASan run, while the explicit leak-support probe terminates with an unsupported-platform message. The report does not interpret that zero exit as evidence of leak freedom. The repaired programs and ownership tests finish with empty ASan stderr.

Task	Main result
1	Payroll: 76000, 28400, 16000
2	One virtual Person; two nonvirtual Persons
3	Constructor/destructor order and initializer warning verified
4	3.5/3.5 defect; 3.5/4.0 repair; slicing confirmed
5	Required total 324.00; pointer search and bounds checked
6	All five return bills and five errors checked

Task 1 - Class design review

The original classes repeat name, ID and basic salary, repeat the DA rule, expose mutable payroll data, use inconsistent names (name/nm and id/code), retain an unused da field, and hide the allowance policy in 0.42. There is no common pay contract, so adding a category requires edits to each processing loop. Inconsistent representations also invite invalid states.

Staff owns private name and ID and declares pure virtual pay() and a virtual destructor. SalariedStaff owns basic and supplies the protected salaryWithDA() helper with a named DA_RATE. Professor adds research, LabAssistant uses salaryWithDA(), and GuestFaculty owns hours and rate. All inheritance is public: each derived object is substitutable for Staff. See payroll.cpp for complete declarations and report/task01-uml.png for UML.

Design defence (under 100 words)

The abstract Staff interface lets one loop calculate every payment through a base pointer. Shared identity belongs to Staff, while the salaried helper keeps the allowance rule in one place. A new category implements pay() and is registered in the collection. Existing payroll classes and the processing loop stay unchanged. Private data prevents unrelated code from silently changing payroll fields.

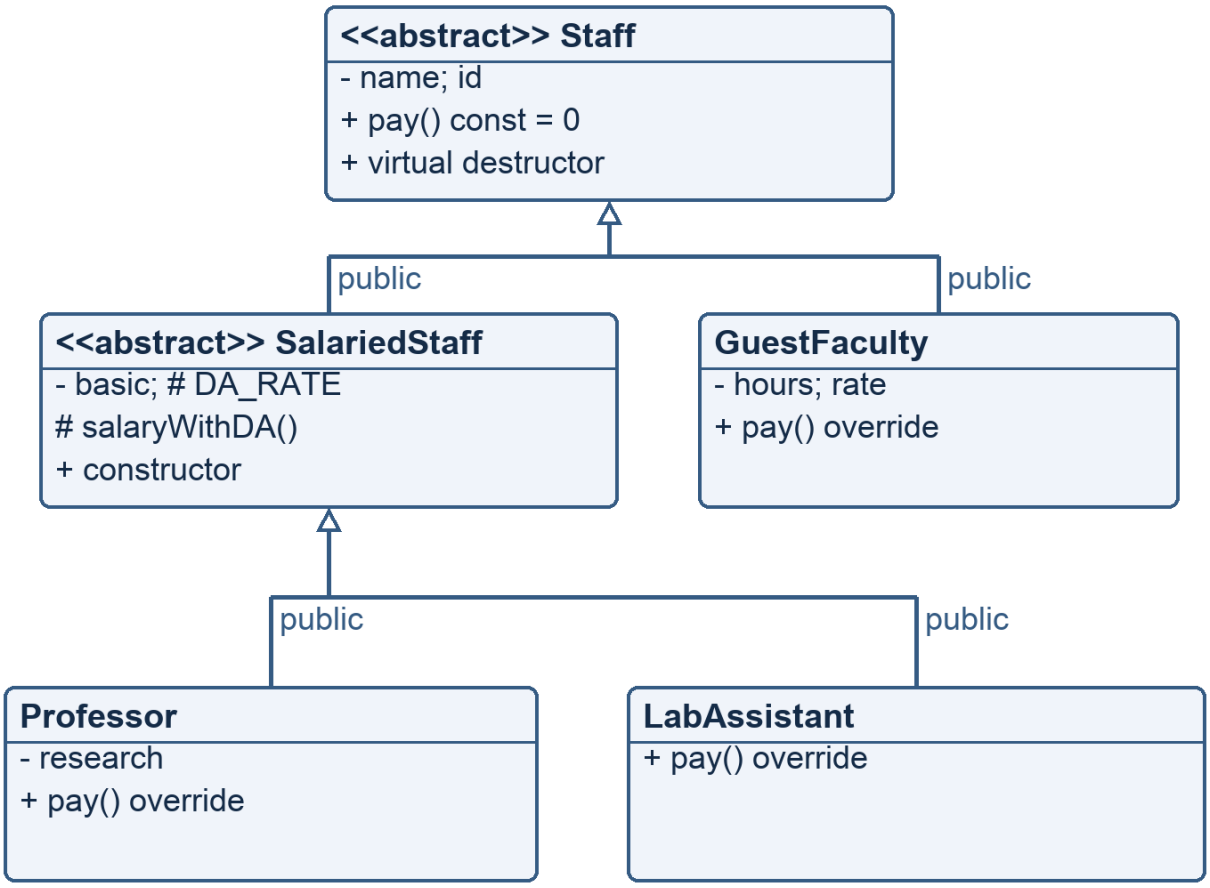
Alternative and complexity

TaggedStaff in payroll.cpp stores a type enum and all possible fields, then switches in pay(). Both implementations take O(1) time per call with three fixed types, O(n) for n staff, and O(1) per-object space under a fixed-size name abstraction (otherwise add name length). The tagged class wastes hours/rate in salaried records and basic/research in guest records. The hierarchy has dynamic-dispatch metadata, commonly one vptr, whose size is ABI-dependent. Actual sizeof results are in evidence/payroll.stdout.txt; sizeof excludes string heap allocations and container storage.

Adding a new tagged category changes at least three places: the enum, switch, and creation/registration site; adding new data may also change the common field declaration (four places). With inheritance, add one new class and update one creation site; zero existing payroll classes and zero processing loops change. Counts exclude serialization and user interfaces, which would require registration in either design. Assertions independently check 76000, 28400, 16000 and zero hours.

UML class diagram

Task 1 | Payroll hierarchy



Hollow triangles point to the base class.
+ public # protected - private

The diagram summarizes the executable declarations. The corresponding .cpp files contain complete class definitions, constructors, access specifiers and method bodies.

Selected observed output

Task01/evidence/payroll.stdout.txt

```
1 Anu 76000.00
2 Bala 28400.00
3 Chitra 16000.00
sizeof Professor=56 LabAssistant=48 GuestFaculty=56 TaggedStaff=72
```

Task 2 - University people management

2.1 Relationships and inheritance

Student is a Person and Staff is a Person: public, virtual single inheritance in each branch. Together these form hierarchical inheritance. TeachingAssistant is both Student and Staff: public multiple inheritance, forming a hybrid diamond with the shared Person. ResearchScholar is a Student: public single inheritance, and Person -> Student -> ResearchScholar is multilevel inheritance. Person has an Address; Student has a Course: composition. A phone is a Person value, and a fellowship is a ResearchScholar value. A teaching assistant's two roles represent one university identity.

2.2 Diamond resolution

Without virtual inheritance, TA contains two Person subobjects. Student::Person and Staff::Person can disagree on name and ID, and unqualified Person member access and conversion to Person* are ambiguous. Virtual inheritance shares one Person. TA, as the most-derived class, initializes it. A Student or Staff constructed by itself initializes its own virtual Person. The deliberately ambiguous id expression is isolated in diagnostics/ambiguous.cpp.txt; its actual compiler error is retained.

2.3 Class design and access

relationships.cpp contains the complete independent declarations and executable demonstration. All identity fields, Address, programme, CGPA, Course, department, salary and fellowship are private; public accessors expose selected identity fields. All is-a edges are public, so external code can use a Person interface. Protected or private inheritance would restrict this substitutability. Constructors pass values through initializer lists. Person has a virtual destructor, and derived destructors use override. The UML is report/task02-uml.png.

2.4 Visibility of Person members in Student

Person member	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	inaccessible directly	inaccessible directly	inaccessible directly

Private members remain present and can be reached through permitted Person functions; inheritance never makes them directly accessible to Student. Only the public result is exposed to ordinary callers. Protected members are available to Student and eligible descendants, subject to the C++ protected access rules.

Prediction and verification

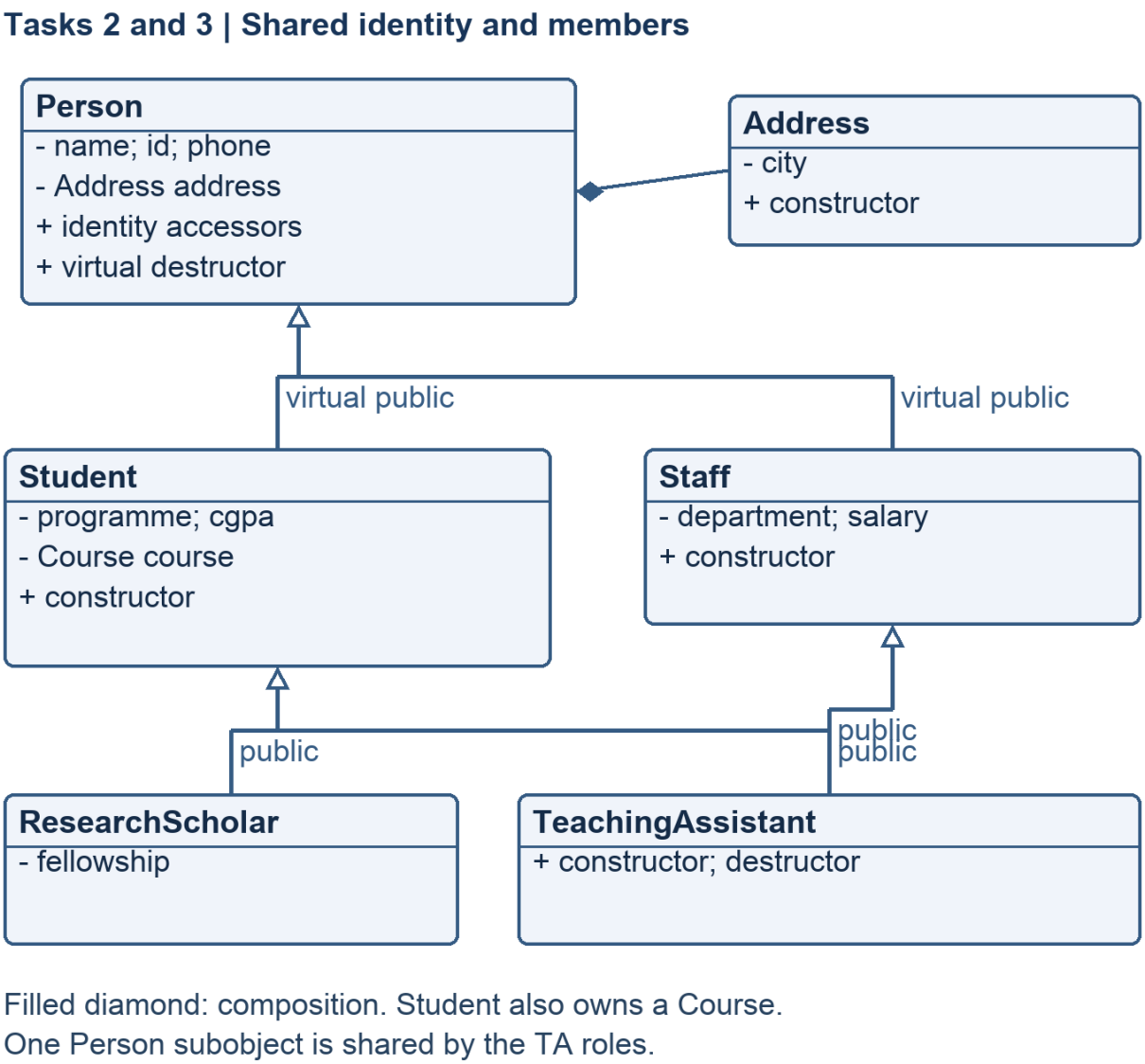
The saved prediction for the supplied virtual program is `Person Student Staff TA` followed by newline. For the nonvirtual variant it is `Person Student Person Staff TA` followed by

newline, with no required compiler diagnostic for construction alone. Both exact stdout comparisons passed, including the trailing space. No differences occurred. Person prints once in the virtual version because the most-derived TA owns one shared virtual base; it prints twice in the nonvirtual version because each branch has a separate base. Removing virtual alone does not cause a compiler error. The separate ambiguous-member fixture does cause an error.

Composition alternative and complexity

`struct ComposedTA { Student student; Staff staff; };` contains two Person subobjects, even though Student and Staff each inherit Person virtually: virtual sharing does not cross complete member objects. Read names with `ta.student.getName()` and `ta.staff.getName()` and define a synchronization policy, or move the common identity into a separate shared owner. The inheritance version uses `ta.getName()` unambiguously. Both accesses are O(1) for references; virtual-base adjustment can add a constant ABI-dependent load. Copying a name costs O(L) in its length. Both designs are O(1) object space under fixed-length fields, but the measured virtual TA is 192 bytes versus ComposedTA 280 bytes; Person is 88 bytes. These sizes include the Task 3 Address/Course members, not string heap allocations. A pair of nonvirtual role members totals 264 bytes. Virtual-base metadata can offset some savings; these numbers are specific to the tested ABI.

UML class diagram



The diagram summarizes the executable declarations. The corresponding .cpp files contain complete class definitions, constructors, access specifiers and method bodies.

Selected observed output

Task02/evidence/predict_virtual.stdout.txt

```
Person Student Staff TA
```

Task02/evidence/predict_nonvirtual.stdout.txt

```
Person Student Person Staff TA
```

Task 3 - Constructors, member objects and lifetime

3.1 Base constructor arguments

hierarchy.cpp implements Student, Staff and TeachingAssistant with initializer lists. Student initializes Person(name,id,phone,city) and its programme, CGPA and Course. Staff initializes Person, department and salary. TA explicitly initializes Person first, then Student and Staff. Intermediate Person initializers are ignored while constructing TA. Otherwise two branches could supply conflicting identity arguments; C++ gives responsibility to the most-derived class. ResearchScholar also directly initializes Person. No default Person constructor is needed.

3.2 Composition

Person declares an Address address member constructed from city. Student declares a Course course member constructed from its title. These are member objects (has-a), not nested class declarations or inherited roles. Their constructors and destructors print messages to make the lifetime visible. The separate supplied complete.cpp initializes the Person base, regNo, then home Address. The pasted assignment's comment and Address data declaration were separated onto intended lines; the TODO replacement is the only behavioural addition.

3.3 Observed order

Student: Address, Person body, Course, Student body; destruction is Student body, Course, Person body, Address. Staff: Address, Person, Staff; destruction is Staff, Person, Address. TA: Address, shared Person, Course, Student, Staff, TA; destruction is TA, Staff, Student, Course, Person, Address. Full messages with values are in evidence/hierarchy.stdout.txt.

Virtual bases are constructed first (depth-first left-to-right across base declarations), then direct nonvirtual bases in declaration order, then member objects in declaration order, and finally the derived constructor body. A Person's Address is constructed before the Person body. Destruction reverses completed construction: derived destructor body, members in reverse order, direct bases in reverse order, virtual bases last. The textual order in an initializer list does not change this.

3.4 Nonvirtual experiment

Compiling the same source with -DNONVIRTUAL removes virtual inheritance. TA then constructs Student's Address/Person, Course/Student, Staff's Address/Person, Staff and TA. Destruction reverses this. Observed sizeof(TA) changes from 192 to 264 bytes; Person=88, Student=152/144 and Staff=128/120 (virtual/nonvirtual). The nonvirtual version duplicates Person but omits virtual inheritance metadata in each role. sizeof is ABI-specific, not a C++ guarantee.

Supplied prediction and completed code

Prediction: `Vehicle Engine Car | ~Car ~Engine ~Vehicle` followed by newline. Actual stdout matches exactly. Vehicle is the direct base; Engine is a member. The Car destructor body precedes the member and base destructors. The supplied TODO program prints exactly `192311001 Kavya (19) Salem-636001` followed by newline. A separately compiled initializer list ordered home, regNo, Person emits -Wreorder-ctor; it still constructs Person, regNo, home. Full warning and successful output are in diagnostics/evidence.

Dynamic Address alternative

address_pointer.cpp demonstrates raw owning Address*, allocated in the constructor and deleted in the destructor. A deep-copy constructor allocates an independent Address, a move constructor transfers ownership and nulls the source, and copy-and-swap assignment handles copying, self-assignment and exceptions. An implicit copy would alias the same allocation and risk double deletion. Constructor allocation may throw; a failed new does not create a completed owner. The by-value assignment parameter is constructed before the noexcept body; a copy allocation failure leaves the target unchanged. The null moved-from state is explicitly supported.

Member storage avoids a separate heap allocation, pointer indirection and custom ownership functions. Pointer storage adds sizeof(Address*) to Student and a separately allocated sizeof(Address) payload plus allocator overhead; it may reduce sizeof(Student), but does not reduce total live storage automatically. On this 64-bit target the pointer alone is 8 bytes. Construction/copy costs O(L) for an L-character address, plus allocator cost A for the pointer version; do not assume the allocator has a guaranteed O(1) worst case. Fixed short addresses are commonly treated as O(1). Destruction releases member or heap storage; the pointer version also deallocates. ASan and assertions passed independent copy, assignment, self-assignment, move and empty-copy cases.

Selected observed output

Task03/evidence/complete.stdout.txt

```
192311001 Kavya (19) Salem-636001
```

Task03/evidence/predict.stdout.txt

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Task03/evidence/hierarchy.stdout.txt

```
Student: Address(Salem) Person(Kavya) Course(OOP) Student ~Student
~Course ~Person ~Address
Staff: Address(Chennai) Person(Bala) Staff ~Staff ~Person ~Address
TeachingAssistant: Address(Salem) Person(Anu) Course(OOP) Student
Staff TA ~TA ~Staff ~Student ~Course ~Person ~Address
sizeof Person=88 Student=152 Staff=128 TA=192 ComposedTA=280
```

Task03/evidence/hierarchy_nonvirtual.stdout.txt

```
Student: Address(Salem) Person(Kavya) Course(OOP) Student ~Student
~Course ~Person ~Address
Staff: Address(Chennai) Person(Bala) Staff ~Staff ~Person ~Address
TeachingAssistant: Address(Salem) Person(Anu) Course(OOP) Student
Address(Salem) Person(Anu) Staff TA ~TA ~Staff ~Person ~Address
~Student ~Course ~Person ~Address
sizeof Person=88 Student=144 Staff=120 TA=264 ComposedTA=264
```

Task 4 - Pointers to derived classes: forensics

4.1 Defects

Account::rate is nonvirtual, so Account* selects Account::rate at compile time and both calls return 3.5. Deleting a SeniorAccount through an Account* whose destructor is nonvirtual is undefined behavior. In the observed run the derived destructor was skipped, so its allocated history was not released. The language does not guarantee that particular symptom. `Account copy = SeniorAccount()` copies only the Account subobject, losing SeniorAccount's state and dynamic behavior (slicing). The temporary SeniorAccount is still destroyed normally at the semicolon.

4.2 Actual evidence and limitation

broken.cpp adds destructor messages to the supplied example and is run in an isolated process with -fsanitize=address -g -fno-omit-frame-pointer. The observed rates are 3.5 and 3.5; the two pointer deletions print only ~Account. The later temporary used for slicing prints ~SeniorAccount and ~Account, followed by Sliced rate=3.5 and the copy's ~Account. That later ~SeniorAccount does not mean the earlier heap senior was released.

ASan exited 0 with empty stderr in this environment: that does not prove correct deletion or absence of leaks. A separate ASAN_OPTIONS=detect_leaks=1 probe terminates with `AddressSanitizer: detect_leaks is not supported on this platform.` (run status -6 in subprocess). Consequently a leak-detector report of the unfreed 100 doubles could not be obtained locally. The history allocation requests 100*sizeof(double)=800 bytes, excluding allocator overhead. The observed destructor trace and the nonvirtual-delete rule establish the ownership defect; the tool did not detect it. Raw stdout/stderr, flags and exit classifications are retained separately.

4.3 Repair

repaired.cpp makes rate virtual and the base destructor virtual, and marks derived functions override. It demonstrates both correctly deleted raw Account pointers and vector<unique_ptr<Account>>. Senior history is an array member, so it is released automatically. A reference to the senior account preserves its dynamic type. Correct rates are 3.5 and 4.0; each senior destruction now prints ~SeniorAccount then ~Account. ASan exits 0 with empty stderr for the repair; local leak detection remains unsupported.

4.4 Prevention

Use a public virtual destructor for a base intended for polymorphic deletion. Declare overriding operations with override, and design ownership around RAII instead of manual owning raw pointers. Preserve polymorphic objects through references or owning base pointers rather than by-value base containers. A polymorphic base can additionally prohibit copying to make slicing harder.

Prediction

Saved predicted output: `3.5 4 3.5` followed by newline. Actual output matches. p->rate() uses the static Account type because rate is nonvirtual. p->vrate() dispatches at run time to

SeniorAccount. copy.vrate() dispatches on a complete Account object after slicing and returns 3.5. Default stream formatting prints 4 rather than 4.0.

unique_ptr alternative and complexity

The implemented vector<unique_ptr<Account>> owns each object and releases it during destruction, including exception unwinding. Processing n accounts takes O(n); each fixed-size rate call takes O(1). Container storage is O(n), plus the same underlying objects as an owning raw-pointer design. A default-deleter unique_ptr commonly occupies one pointer but this size is not guaranteed. Vector capacity and allocator bookkeeping add overhead in either heap-based collection.

unique_ptr alone does not make rate virtual and does not repair a nonvirtual base destructor: both wrong dispatch and undefined polymorphic deletion remain possible. The vector of unique_ptr prevents value slicing within that container, but an explicit Account copy elsewhere still slices. Smart ownership and a correct virtual interface solve different problems.

Selected observed output

Task04/evidence/predict.stdout.txt

```
3.5 4 3.5
```

Task04/evidence/broken_asan.stdout.txt

```
3.5
3.5
~Account
~Account
~SeniorAccount
~Account
Sliced rate=3.5
~Account
```

Task04/evidence/repaired_asan.stdout.txt

```
3.5
4
~Account
~SeniorAccount
~Account
RAII alternative
3.5
4
~SeniorAccount
~Account
~Account
```

Task 5 - Object access and this

5.1 Four ways

cart_demo.cpp reaches one stack Cart directly as c.add(...), through Cart* p=&c and p->add(...), and through Cart& ref=c and ref.setDiscount(...). A fourth Cart is created with new, reached with ->, and released exactly once by delete. add returns Cart& using return *this; setDiscount uses this->discount=discount to distinguish the member from its parameter. The supplied skeleton is completed separately in complete.cpp and produces exactly 324.00 followed by newline: (60+120+180)*0.90=324.

5.2 Highest total

highest scans an array of Cart pointers by index. It skips null entries, retains the first cart with the largest total, and returns nullptr for an empty array or an array with no objects. No pointer is dereferenced unless non-null. Embedded tests cover an empty array, a null entry, a full ten-item cart (the eleventh add is ignored), and 100% discount. The expanded demo gives 162.00 for the shared stack cart and 180.00 for the highest cart. A tie keeps the first encountered cart.

5.3 Comparison

Access	Lifetime and safety	Suitable use
Named object	automatic scope lifetime; cleanup automatic	ordinary local ownership
Borrowed pointer	nullable; may dangle; does not own	optional access and pointer collections
Reference	must bind to an object; may still dangle	required alias or parameter
new/delete pointer	explicit dynamic lifetime; leaks/double delete possible	raw allocation demonstration; prefer unique_ptr for owning production code

this is an implicit pointer to the current object in a nonstatic member function. In this Cart's mutating methods it has type Cart*; in total() const it has type const Cart*. *this is an lvalue referring to that same Cart, so returning Cart& keeps all chained operations on one object. Returning Cart by value would copy the whole Cart. Then c.add("Rice",60) would modify c, but later chained Dal and discount operations would modify temporaries. The later p->add("Oil",180) would leave c with Rice and Oil and no discount, so the original main would print 240.00.

vector alternative and complexity

Pseudocode: STORE Cart values in vector; SET best to no index; FOR each index i, IF best absent OR carts[i].total()>carts[best].total() THEN SET best to i; END IF; END FOR; RETURN best. Initializing vector<Cart> from existing lvalues copies Cart objects; emplacement or moves can avoid those initial copies. A pointer array stores one pointer per cart, but the actual Cart

objects still occupy storage elsewhere. Both total memory requirements are O(n) for n carts under the fixed ten-item bound; vectors generally improve locality and manage ownership.

highest is O(n*m) if m items must be summed, hence O(n) here because m<=10, with O(1) extra search space. Copying string items additionally depends on total character count L. Vector growth can reallocate and invalidate all pointers, references and iterators into its elements. An index can be used after reallocation (but erasure can shift indices); reserve avoids reallocations only until capacity is exceeded. A raw pointer array preserves pointee addresses only while the separately owned objects remain alive. The supplied skeleton intentionally keeps its original discount and price semantics; arbitrary negative prices or out-of-range discounts are not newly validated.

Selected observed output

Task05/evidence/complete.stdout.txt

```
324.00
```

Task05/evidence/cart_demo.stdout.txt

```
Direct/pointer/reference total=162.00
Highest=180.00
Empty array, null entry, capacity and 100% discount checks passed
```

Task 6 - Vehicle rental fleet

Stage 1: Entities and testable requirements

Vehicle owns registration, daily rate, availability and agreed days. Bicycle has the basic daily charge. MotorVehicle adds fuel/charge percentage and a ServiceRecord member holding the due date. TwoWheeler adds a per-litre price. Car adds seats and a per-litre price. ElectricCar adds battery kWh and a per-kWh price and overrides the fuel charge. ElectricBus is a later extension with an extra daily cleaning charge. This interpretation treats charge level as a percentage and usage supplied at return as litres for combustion vehicles or kWh for electric vehicles.

Testable statements: list every type through Vehicle pointers; rent each available vehicle for positive days; reject a second rent; reject unknown registrations and invalid days; reject a return without an active rental; reject negative usage without changing rental state; return every type and restore availability; calculate an overdue return; and add ElectricBus without editing existing vehicle classes. input.txt is actually consumed by fleet.cpp and exercises these states.

Stage 2: Hierarchy

Vehicle is abstract through pure virtual charge() and kind(), with a public virtual destructor. Vehicle -> Bicycle and Vehicle -> MotorVehicle form hierarchical inheritance. MotorVehicle -> TwoWheeler and MotorVehicle -> Car branch again. Car -> ElectricCar -> ElectricBus is multilevel inheritance. Each individual edge is public single inheritance; no multiple or virtual inheritance is needed. MotorVehicle has a ServiceRecord member. See report/task06-uml.png.

Stage 3: Decisions and assumptions

Identity, rental state and transition checks belong to Vehicle so all types follow the same rent/return protocol. charge() varies and is virtual. rent and returnVehicle share fixed validation and billing workflow and call the virtual charge(). kind() supports listing. Data is private, with protected baseCharge() for reuse. Five stack objects own their lifetimes and an array<Vehicle*,5> supplies the required base-pointer collection. Its entries are non-owning and remain valid for the whole processing loop.

Pricing is an explicit demonstration policy because the brief gives no exact formulas: Bicycle=days*daily rate; TwoWheeler and Car add litres*price; ElectricCar adds kWh*price; ElectricBus also adds 50 per day for cleaning. A late return bills actual elapsed days and adds 25% of daily rate for each overdue day. Daily rates are 100,400,1000,1200,3000; litre price is 100 and kWh price is 10. Battery capacity, service date and level are recorded; no automatic physical fuel depletion or date scheduling is assumed. Car's per-litre field is zero in ElectricCar and is unused by its overridden charge, a small reuse tradeoff. A larger system could separate propulsion pricing by composition.

Stage 4: Classes and pseudocode

fleet.cpp contains full declarations with access specifiers and implementations. Pseudocode follows explicit INPUT/OUTPUT, assignment, selection and iteration blocks:

```
ALGORITHM RentVehicle
INPUT Fleet, Registration, Days
OUTPUT Success or error
SET V TO the matching pointer found by scanning Fleet
IF V is absent OR Days <= 0 OR V is already rented THEN
    OUTPUT error
ELSE
    SET V.agreedDays TO Days
    SET V.rented TO TRUE
    OUTPUT success
END IF
END ALGORITHM

ALGORITHM ReturnVehicle
INPUT Fleet, Registration, ElapsedDays, Usage
OUTPUT Bill or error
SET V TO the matching pointer found by scanning Fleet
IF V is absent OR V is not rented OR ElapsedDays <= 0
   OR Usage < 0 OR Usage is not finite THEN
    OUTPUT error
ELSE
    SET LateDays TO MAX(0, ElapsedDays - V.agreedDays)
    SET Bill TO V.charge(ElapsedDays, Usage)
                + LateDays * V.dailyRate * 0.25
    SET V.rented TO FALSE
    SET V.agreedDays TO 0
    OUTPUT Bill
END IF
END ALGORITHM
```

Stage 5: Demonstration and independent results

All five types rent for two agreed days and return after three. Bicycle: 3*100+25=325. TwoWheeler: 3*400+2*100+100=1500. Car: 3*1000+5*100+250=3750. ElectricCar: 3*1200+20*10+300=4100. ElectricBus: 3*3000+50*10+3*50+750=10400. The runner checks all five values and five expected errors. An on-time Bicycle return later bills 100. The final list shows all five available. Invalid negative usage leaves the active rental intact so a valid return can follow.

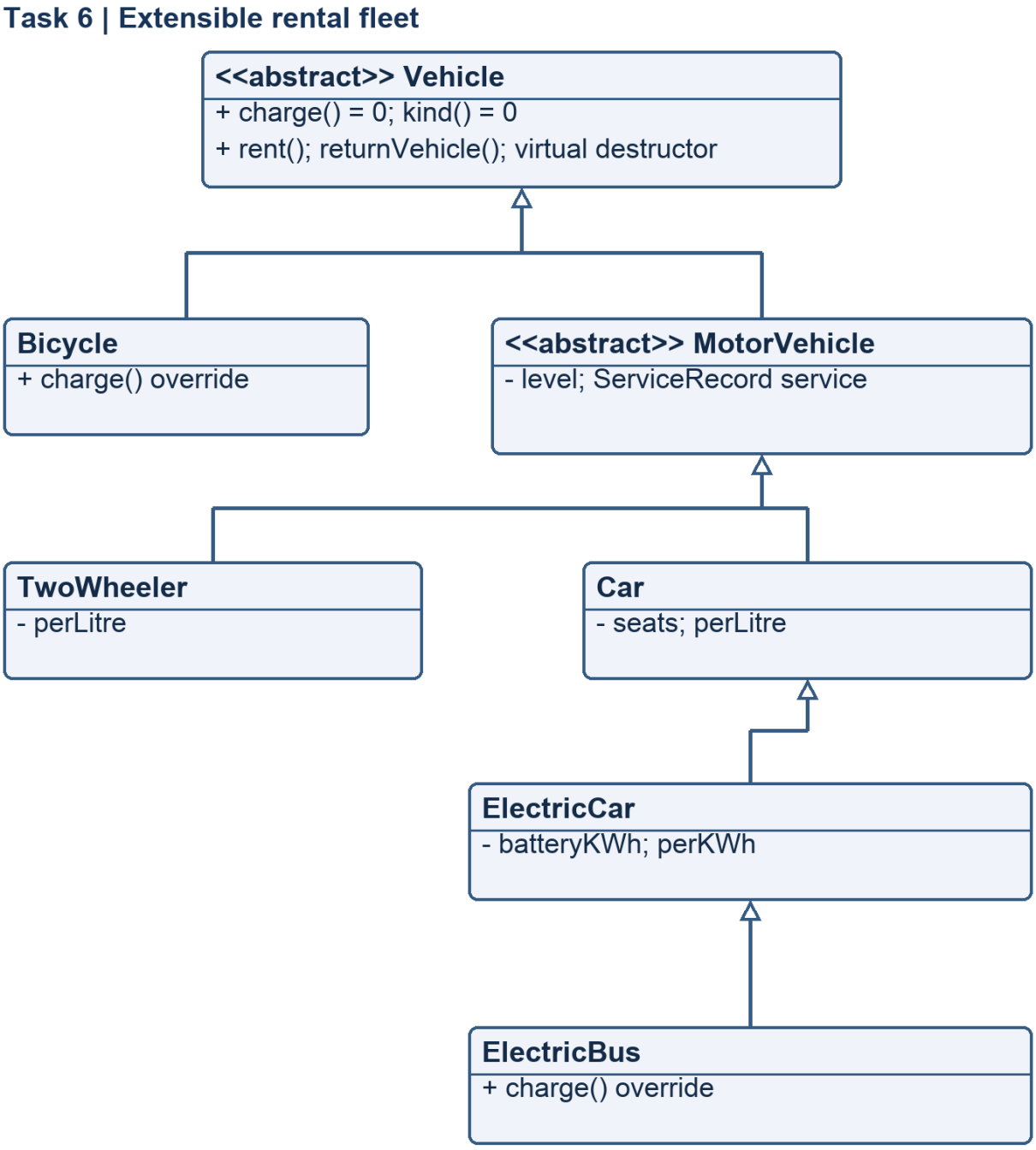
ElectricBus is implemented as an added subclass overriding charge() and kind(). Only the demo construction and pointer-array registration are expanded; existing classes and processing functions are unchanged. See evidence/fleet.stdout.txt for the complete consumed-input transcript.

Alternative and complexity

tagged.cpp implements a single TaggedVehicle with an enum and switch. It computes the electric bus charge before the late fee as 9650 and checks it independently. charge is O(1) for both a fixed switch and a virtual call. Search/rent/return takes O(n) due to linear registration lookup, while listing takes O(n); storage is O(n) plus registration/date string lengths. An individual fixed-field object uses O(1) space, excluding dynamic string buffers.

The tagged representation keeps seats, battery, both pricing units, service date and level even in bicycles. The hierarchy stores only fields along a type's base chain, plus implementation-dependent virtual dispatch metadata (commonly one vptr for this single hierarchy). The Car/ElectricCar reuse tradeoff above leaves one unused field, so it is not literally zero wasted space. Actual TaggedVehicle sizeof is in evidence/tagged.stdout.txt; sizeof excludes string heap payloads. Adding a type in the tagged approach changes enum, switch and creation/registration (at least three places); a new data shape may also change common fields. In the hierarchy, add one class and its creation/registration while changing zero existing class definitions and zero generic loops. No claim is made that every application-level factory or serializer needs no registration.

UML class diagram



The diagram summarizes the executable declarations. The corresponding .cpp files contain complete class definitions, constructors, access specifiers and method bodies.

Selected observed output

Task06/evidence/fleet.stdout.txt

```
B1 Bicycle available
T1 TwoWheeler available
C1 Car available
E1 ElectricCar available
EB1 ElectricBus available
RENT B1 OK
RENT T1 OK
RENT C1 OK
RENT E1 OK
RENT EB1 OK
ERROR B1 unavailable or invalid rental days
RETURN B1 charge=325.00
RETURN T1 charge=1500.00
RETURN C1 charge=3750.00
RETURN E1 charge=4100.00
RETURN EB1 charge=10400.00
ERROR B1 invalid return
ERROR X1 unknown registration
ERROR B1 unavailable or invalid rental days
RENT B1 OK
ERROR B1 invalid return
RETURN B1 charge=100.00
B1 Bicycle available
T1 TwoWheeler available
C1 Car available
E1 ElectricCar available
EB1 ElectricBus available
```

Task06/evidence/tagged.stdout.txt

```
Bus charge before late fee=9650 sizeof TaggedVehicle=104
```

Implementation index and verification discussion

Task	Source files
1	Task01/payroll.cpp
2	Task02/relationships.cpp; predict_virtual.cpp; predict_nonvirtual.cpp; diagnostics/ambiguous.cpp.txt
3	Task03/hierarchy.cpp; complete.cpp; predict.cpp; address_pointer.cpp; diagnostics/reordered.cpp.txt
4	Task04/broken.cpp; repaired.cpp; predict.cpp
5	Task05/complete.cpp; cart_demo.cpp
6	Task06/fleet.cpp; tagged.cpp; input.txt

The supplied TODO skeletons preserve their surrounding behavior. Only intended line breaks lost in the pasted comment formatting were restored. Expanded tests are separate executables. The deliberate unsafe and diagnostic programs are isolated in the runner and are not examples of valid production ownership.

For viva preparation: explain why TA initializes Person directly; trace Address and Course lifetimes; distinguish static binding from dynamic dispatch; explain why a unique_ptr still needs a virtual base destructor; trace reference-return chaining; and add one new vehicle subclass by implementing the charge and kind contract.

Complexity assumes a fixed set of types and fixed-size numeric data unless string length or item count is stated. sizeof measurements are specific to this ABI and exclude separately allocated string payloads, allocator metadata and collection capacity. No timing ranking is inferred solely from Big-O or object sizes.

Specification basis: the supplied Assignment 4 - CO4 brief. All outputs and sizes quoted here come from the accompanying programs and logs. Pricing policies not given in the brief are explicitly identified as demonstration assumptions.